

Mastering @ConditionalOnProperty

This is one of the **most powerful but misunderstood annotations in Spring Boot**.

It decides **which Beans even exist** in your application.

Why does this concept exist?

In real projects:

- Hundreds of Beans exist
- Not all are needed always
- Some are environment-specific
- Some are feature-specific

Without conditions:

- All Beans load
- Startup slows
- Memory increases
- Conflicts occur

We need **conditional bean creation**.

Why @ConditionalOnProperty Exists ?

Problem:

Spring by default eagerly creates Beans.

Result:

IoC Container becomes cluttered.

Example:

You support both:

- MySQL
- NoSQL

Without conditions → both connections load.

This creates:

- Conflicts
- Wasted resources
- Wrong runtime behavior

What is @ConditionalOnProperty ?

It is a **Spring Boot** conditional annotation.

It tells Spring:

“Create this Bean only if a property matches a value.”

It links:

application.properties → Bean existence

Where It Fits in Spring Boot Architecture ?

Flow:

```
pom.xml
↓
Auto-configuration classes
↓
@ConditionalOnProperty checks Environment
↓
If condition matches → Bean created
Else → Bean skipped
```

This happens **during ApplicationContext startup**.

Real-World Use Case

Scenario

Company migrating from **MySQL** → **NoSQL**

We want:

- Only one DB connection active
- Switchable via config

Base Setup (Without Condition — Problem)

Interface

```
public interface DatabaseConnection {  
    void connect();  
}
```

MySQL

```
@Component  
public class MySQLConnection implements DatabaseConnection {  
    public void connect() {  
        System.out.println("Connected to MySQL");  
    }  
}
```

NoSQL

```
@Component  
public class NoSQLConnection implements DatabaseConnection {  
    public void connect() {  
        System.out.println("Connected to NoSQL");  
    }  
}
```

Injection

```
@Autowired  
private DatabaseConnection connection;
```

✗ Result:

No qualifying bean of type DatabaseConnection

2 beans found: MySQLConnection, NoSQLConnection

Applying @ConditionalOnProperty

MySQL Bean

```
@Configuration
public class MySQLConfig {

    @Bean
    @ConditionalOnProperty(
        prefix = "mysqlconnection",
        value = "enabled",
        havingValue = "create",
        matchIfMissing = false
    )
    public DatabaseConnection mysqlConnection() {
        return new MySQLConnection();
    }
}
```

NoSQL Bean

```
@Configuration
public class NoSQLConfig {

    @Bean
    @ConditionalOnProperty(
        prefix = "nosqlconnection",
        value = "enabled",
        havingValue = "create",
        matchIfMissing = false
    )
    public DatabaseConnection nosqlConnection() {
        return new NoSQLConnection();
    }
}
```

application.properties Cases

Case 1

```
mysqlconnection.enabled=create  
nosqlconnection.enabled=dontcreate
```

Result:

- ✓ MySQL Bean created
- ✗ NoSQL Bean skipped

Case 2

```
mysqlconnection.enabled=dontcreate  
nosqlconnection.enabled=create
```

Result:

- ✓ NoSQL Bean created
- ✗ MySQL Bean skipped

Case 3 — No Property In Application Properties + matchIfMissing = true

```
@ConditionalOnProperty(  
    prefix="nosqlconnection",  
    value="enabled",  
    havingValue="true",  
    matchIfMissing=true  
)
```

Result:

- ✓ Bean created by default

Case 4 — No Property In Application Properties + matchIfMissing = false

Result:

- ✗ Bean NOT created

Why @Autowired(required = false) Is Needed

```
@Autowired(required = false)
private DatabaseConnection connection;
```

Without this:

If no Bean is created → App crashes.

With this:

Application starts safely with bean as null if not created.

Combined Case: Two Conditional Beans

mysql.enabled	nosql.enabled	Result
create	dontcreate	MySQL
dontcreate	create	NoSQL
dontcreate	dontcreate	No Bean
create	create	Conflict

How Spring Executes This Internally

Startup Flow:

1. SpringApplication.run()
2. Loads Environment (properties)
3. Loads auto-config classes
4. Sees @ConditionalOnProperty
5. Reads Environment
6. Matches values
7. Registers Bean or skips

This happens **before Beans exist**.

Advantages

- Feature toggling
- Faster startup
- Cleaner IoC container
- Environment-based behavior

Disadvantages

- Misconfiguration risk
- Debug complexity
- Harder local testing

Differences Table

Annotation	Purpose
@Profile	Environment based
@ConditionalOnProperty	Config based
@ConditionalOnClass	Classpath based

Hidden Traps

- Property typo = Bean missing
- Both Beans ON = startup failure
- Proxy injection confusion
- Silent behavior changes

Interview Perspective

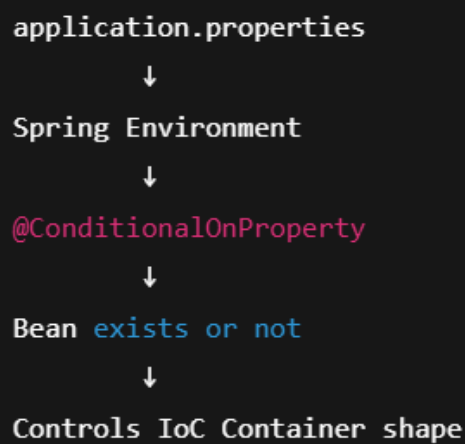
They test:

- ✓ Do you understand config-driven architecture
- ✓ Do you know startup internals
- ✓ Can you design flexible systems

Most Asked Interview Questions

1. What is `@ConditionalOnProperty`?
→ Property-based Bean creation control.
2. When is it evaluated?
→ During context startup.
3. Difference from `@Profile`?
→ Property vs Environment based.
4. What is `matchIfMissing`?
→ Default behavior if property absent.
5. Why use `required=false`?
→ Prevent startup failure.
6. Can it be used on classes?
→ Yes.
7. What happens if both Beans match?
→ Conflict exception.
8. Is this Spring or Spring Boot?
→ Spring Boot only.
9. Where is logic implemented?
→ `spring-boot-autoconfigure` jar.
10. Can it override auto-config beans?
→ Yes.

One-Page Mental Model



Enterprise Relevance

Used heavily in:

- Microservices
- Feature flags
- Multi-DB systems
- Cloud-native apps

Summary as a Story

1. You create a Spring Boot app.
2. You add MySQL & NoSQL dependencies.
3. Spring Boot starts.
4. It loads `application.properties`.
5. Environment is built.
6. Before creating Beans, Spring reads:

```
mysqlconnection.enabled=create
```

7. Spring sees: "MySQL Bean condition matched."
8. It registers `MySQLConnection`.
9. It skips `NoSQLConnection`.
10. IoC Container is shaped **by configuration**.

Your app behavior changed — without touching code.

That is **configuration-driven architecture**.

In real systems, this is how:

- Features are toggled
- Databases are switched
- Integrations are enabled/disabled
- Environments behave differently

This is configuration-driven architecture.

Core Definitions

Term	Meaning
Bean	Object managed by Spring
IoC Container	ApplicationContext
Environment	All properties loaded
@Configuration	Class that defines Beans
@Bean	Method that creates a Bean
@ConditionalOnProperty	Property-based Bean filter